LOW-LEVEL DESIGN (LLD) – RAG Customer Support Assistant

1. Introduction

The Low-Level Design (LLD) defines the **internal working of the system**, including modules, data structures, workflows, and logic.

This system is designed to:

- Process PDF knowledge base
- Retrieve relevant information using embeddings
- Generate contextual answers
- Apply workflow-based decision logic
- Escalate to human when required

2. Module-Level Design

The system is divided into the following modules:

1. Document Processing Module

- Loads PDF file
- Extracts text content

2. Chunking Module

- Splits text into smaller chunks
- Uses chunk size ~300 and overlap ~50

3. Embedding Module

- Converts text chunks into embeddings
- Uses semantic vector representation

4. Vector Storage Module

- Stores embeddings in ChromaDB
- Enables fast similarity search

5. Retrieval Module

- Converts query into embedding
- Finds most relevant chunks

6. Query Processing Module

- Handles user input
- Passes query through system

7. Graph Execution Module (LangGraph)

- Controls flow between nodes
- Manages execution state

8. HITL Module

- Handles human escalation
- Returns human-generated response

3. LLD Architecture Diagram

```
        +----------------------+
        |   User Interface     |
        +----------+-----------+
                   |
                   v
        +----------------------+
        | Query Processing     |
        | Module               |
        +----------+-----------+
                   |
                   v
        +----------------------+
        | Graph Execution      |
        | Module (LangGraph)   |
        +----------+-----------+
                   |
                   v
    +-----------------------------------+
    | Retrieval Module                  |
    | (Query → Embedding → Search)      |
    +----------+------------------------+
               |
               v
    +------------------------------+
    | Vector Storage (ChromaDB)    |
    +----------+-------------------+
               |
               v
    +------------------------------+
    | Retrieved Chunks             |
    +----------+-------------------+
               |
               v
    +------------------------------+
    | LLM / Response Generator     |
    +----------+-------------------+


               |
```

```
        +--------+---------+
        |                  |
        v                  v
+------------------+   +----------------------+
| Output Module    |   | HITL Module          |
| (Final Answer)   |   | (Human Escalation)   |
+------------------+   +----------------------+
```

---

4. Data Structures

1. Document Structure

```
{
  "text": "Full extracted text",
  "metadata": {
    "source": "PDF file"
  }
}
```

2. Chunk Structure

```
{
  "chunk_id": 1,
  "content": "Partial text",
  "metadata": {
    "page_number": 2
  }
}
```

3. Embedding Structure

```
{
  "vector": [0.123, 0.456, ...],
  "chunk_id": 1
}
```

4. Query-Response Schema

```
{
  "query": "What is refund policy?",
  "retrieved_chunks": [...],
  "response": "Answer text"
}
```

5. State Object (LangGraph)

```
{
  "user_query": "",
  "retrieved_data": "",
  "response": "",
  "status": ""
}
```

5. Workflow Design (LangGraph)

Workflow Diagram

```
+-------------+
| Input Node  |
+------+------+
       |
       v
+-------------+
| Processing  |
| Node        |
+------+------+
       |
       v
+-------------+
| Output Node |
+-------------+
```

Explanation

- **Input Node** → Receives user query
- **Processing Node** → Retrieves data and generates response
- **Output Node** → Displays final result

6. Conditional Routing Logic

Decision Diagram

```
         Retrieved Answer
                  |
     +----------+----------+
     |                     |
Relevant              Not Relevant
     |                     |
     v                     v
Show Answer          Escalate to HITL
```

Conditions

Answer Generation:

- High similarity score
- Relevant context found

Escalation:

- No relevant chunks
- Low confidence
- Complex query

7. HITL (Human-in-the-Loop) Design

Flow Diagram

```
User Query
    ↓
System Processing
    ↓
Low Confidence / No Answer
    ↓
Send to Human Agent
    ↓
Human Generates Response
    ↓
Return to User
```

Explanation

- Triggered when AI fails
- Human provides accurate response
- Improves trust and reliability

8. API / Interface Design

Input

```
{
  "query": "User question"
}
```

Output

```
{
  "response": "Generated answer"
}
```

Flow

User → System → Response

9. Error Handling

Cases:

- Missing PDF → show error
- No relevant chunks → fallback message
- LLM failure → retry or escalate

10. Execution Flow Summary

```
PDF → Load → Chunk → Embed → Store
User Query → Embed → Search → Retrieve → Generate Answer
→ Check Condition → Output / HITL
```